

AA Eksen Firebase Entegrasyon Dokümantasyonu (Next.js & Unity)

Bu doküman, AA Eksen mobil uygulaması için oluşturulan Firebase altyapısının **Next.js (Web)** ve **Unity (Crossword Oyunu)** platformlarına nasıl entegre edileceğini detaylı bir şekilde açıklamaktadır.

1. Genel Bakış: Firebase Yapısı

Projemiz, kullanıcı verileri, haberler, oyunlar ve AI etkileşimleri için **Cloud Firestore**'u ana veritabanı olarak kullanır. Güvenlik için **Firestore Authentication** ve analiz için **Firestore Analytics** entegre edilmiştir.

1.1. Ana Firestore Koleksiyonları

Aşağıdaki şema, Firestore veritabanımızın ana koleksiyon yapısını göstermektedir. Değişken isimleri, mobil uygulamadaki modellerle birebir aynıdır.

```
/
├── users/{userId}
├── news/{newsId}
├── time_tunnel_categories/{categoryId}
├── crossword_categories/{categoryId}
├── crossword_word_groups/{groupId}
├── quiz_categories/{categoryId}
├── quiz_questions/{questionId}
├── daily_tasks/{taskId}
├── badges/{badgeId}
├── ai_questions/{questionId}
```

2. Veri Modelleri (Data Models)

Tüm platformlarda (Mobil, Web, Unity) veri tutarlılığını sağlamak için aşağıdaki veri modellerini referans almalısınız.

2.1. Haber Modeli (news)

Her bir doküman, bir haberi temsil eder.

Koleksiyon: news

Alan Adı	Veri Tipi	Açıklama
id	string (Document ID)	Firestore tarafından otomatik atanan eşsiz ID.
title	string	Haberin başlığı.

Alan Adı	Veri Tipi	Açıklama
content	string	Haberin tam metni.
author	string	Haberi yazan kişi veya kaynak.
publishedAt	Timestamp	Haberin yayınlanma tarihi.
imageUrl	string	Haber görselinin URL'si.
imageCaption	string	Görseli açıklayan kısa metin.
category	string	Haberin kategorisi (örn: "Spor", "Gündem").
location	string	Haberin geçtiği il (örn: "İstanbul").
xpPoints	number	Bu haberi okumanın kazandırdığı XP puanı.
likeCount	number	Beğeni sayısı.
viewCount	number	Görüntülenme sayısı.
shareCount	number	Paylaşılma sayısı.
breaking	boolean	true ise haber manşettir (Breaking News).
newsUrl	string	Haberin web sitesindeki orijinal URL'si (Paylaşım için).
verified	boolean	Haberin AA tarafından doğrulanıp doğrulanmadığı.
createdAt	Timestamp	Dökümanın oluşturulma tarihi.
updatedAt	Timestamp	Dökümanın son güncellenme tarihi.
crosswordWords	array of string	Çengel bulmaca için haberden çıkarılan kelimeler.
quizQuestions	array of map	Haberle ilgili quiz soruları.
... (Diğer AI ve oyun alanları)		

2.2. Zaman Tüneli Kategorisi (time_tunnel_categories)

Her döküman, bir zaman tüneli konusunu temsil eder.

Koleksiyon: time_tunnel_categories

Alan Adı	Veri Tipi	Açıklama
id	string (Doc ID)	Eşsiz kategori ID'si.
title	string	Kategorinin başlığı (örn: "Ekrem İmamoğlu Tutuklanma Süreci").
description	string	Kategorinin kısa açıklaması.
coverImageUrl	string	Kategori için kapak görseli URL'si.

Alan Adı	Veri Tipi	Açıklama
newsIds	array of string	Bu kategorideki haberlerin ID'lerini kronolojik sırada tutar.
active	boolean	true ise kategori uygulamada görünür.
createdAt	Timestamp	Oluşturulma tarihi.
startDate	Timestamp (nullable)	Zaman tünelinin başlangıç tarihi.
endDate	Timestamp (nullable)	Zaman tünelinin bitiş tarihi.

2.3. Çengel Bulmaca Kategorisi (**crossword_categories**)

Koleksiyon: **crossword_categories**

Alan Adı	Veri Tipi	Açıklama
id	string	Eşsiz kategori ID'si.
name	string	Kategorinin adı (örn: "Siyaset", "Ekonomi").
xpPoints	number	Bu kategoriye tamamlamanın kazandırdığı XP.
iconUrl	string	Kategori ikonu URL'si.

2.4. Çengel Bulmaca Kelime Grubu (**crossword_word_groups**)

Koleksiyon: **crossword_word_groups**

Alan Adı	Veri Tipi	Açıklama
id	string	Eşsiz kelime grubu ID'si.
categoryId	string	Hangi kategoriye ait olduğu (crossword_categories ID'si).
words	array of map	10 adet kelime ve ipucunu içeren liste.

words içindeki her bir **map** objesi:

- word**: **string** (Cevap, örn: "MECLİS")
- clue**: **string** (İpucu, örn: "Yasama organı")
- xp**: **number** (Kelimeyi bilmenin kazandırdığı XP)

3. Next.js (Web) Entegrasyonu

Next.js projenizde Firebase'i kullanmak için **firebase** SDK'sını kurmanız ve yapılandırmanız gerekir.

3.1. Kurulum ve Yapılandırma

1. Firebase SDK'sını Yükleyin:

```
npm install firebase
```

2. **Firestore'i Başlatın:** Projenizin kök dizininde bir `firebase.js` (veya `firebase.ts`) dosyası oluşturun. Bu dosyaya Firebase projenizin yapılandırma bilgilerini ekleyin. Bu bilgileri Firebase Console > Proje Ayarları > Genel bölümünden alabilirsiniz.

```
// lib/firebase.js
import { initializeApp, getApps } from "firebase/app";
import { getFirestore } from "firebase/firestore";
import { getAuth } from "firebase/auth";

const firebaseConfig = {
  apiKey: "YOUR_API_KEY",
  authDomain: "YOUR_AUTH_DOMAIN",
  projectId: "aa-eksen-71097",
  storageBucket: "YOUR_STORAGE_BUCKET",
  messagingSenderId: "YOUR_MESSAGING_SENDER_ID",
  appId: "YOUR_APP_ID"
};

// Sunucu tarafında (SSR) yeniden başlatmayı önlemek için kontrol
const app = !getApps().length ? initializeApp(firebaseConfig) :
getApps()[0];
const db = getFirestore(app);
const auth = getAuth(app);

export { db, auth };
```

3.2. Veri Çekme (Örnek: Haber Listesi)

`getStaticProps` veya `getServerSideProps` içinde Firestore'dan veri çekebilirsiniz.

```
// pages/index.js
import { db } from '../lib/firebase';
import { collection, getDocs, query, orderBy, limit } from
'firebase/firestore';

export async function getStaticProps() {
  const newsCol = collection(db, 'news');
  const q = query(newsCol, orderBy('publishedAt', 'desc'), limit(20));
  const newsSnapshot = await getDocs(q);

  const newsList = newsSnapshot.docs.map(doc => ({
    id: doc.id,
    ...doc.data(),
    // Timestamp'i serileştirilebilir formata dönüştür
    publishedAt: doc.data().publishedAt.toDate().toISOString(),
  }));
}
```

```
return {
  props: {
    news: newsList,
  },
  revalidate: 60, // 60 saniyede bir sayfayı yeniden oluştur (ISR)
};
}

// React component'inizde bu veriyi kullanın
function HomePage({ news }) {
  return (
    <div>
      {news.map(item => (
        <div key={item.id}>
          <h2>{item.title}</h2>
          <p>{item.author}</p>
        </div>
      ))}
    </div>
  );
}

export default HomePage;
```

3.3. Zaman Tüneli Entegrasyonu

- **Kategori Listesi (/time-tunnel):** `time_tunnel_categories` koleksiyonundan `active: true` olanları çekin.
- **Detay Sayfası (/time-tunnel/[categoryId]):**
 1. `getStaticPaths` ile tüm kategori ID'leri için yollar oluşturun.
 2. `getStaticProps` içinde, gelen `categoryId` ile ilgili `TimeTunnelCategory` dökümanını çekin.
 3. `newsIds` dizisindeki her bir ID için `news` koleksiyonundan ilgili haberleri çekin. Bu haberleri kronolojik olarak sıralayın.

4. Unity (Crossword Oyunu) Entegrasyonu

Unity projenize Firebase'i entegre etmek için **Firestore Unity SDK**'sını kullanmalısınız.

4.1. Kurulum ve Yapılandırma

1. **Firestore Unity SDK'sını İndirin:** Firestore'in resmi sayfasından **.NET 4.x** için olan SDK'yı indirin. İhtiyacınız olan paketler:
 - `FirestoreUnityPackage`
 - `AuthUnityPackage` (Kullanıcı bazlı veri saklayacaksanız)
2. **SDK'yı Unity'ye Ekleyin:** İndirdiğiniz `.unitypackage` dosyalarını projenize import edin (Assets > Import Package > Custom Package).

3. Yapılandırma Dosyalarını Ekleyin:

- Firebase Console > Proje Ayarları bölümünden Unity için **google-services.json** (Android) ve **GoogleService-Info.plist** (iOS) dosyalarını indirin.
- Bu dosyaları Unity projenizin **Assets** klasörüne sürükleyip bırakın.

4.2. Firebase'i Başlatma (C# Script)

Oyununuzun başlangıcında çalışan bir script (örn: **FirebaseManager.cs**) oluşturun ve Firebase'i başlatın.

```
using UnityEngine;
using Firebase;
using Firebase.Firestore;

public class FirebaseManager : MonoBehaviour
{
    public static Firestore db;

    void Start()
    {
        FirebaseApp.CheckAndFixDependenciesAsync().ContinueWith(task => {
            var dependencyStatus = task.Result;
            if (dependencyStatus == DependencyStatus.Available)
            {
                // Firebase'i başlat
                FirebaseApp app = FirebaseApp.DefaultInstance;
                db = Firestore.DefaultInstance;
                Debug.Log("Firebase başarıyla başlatıldı.");
            }
            else
            {
                Debug.LogError($"Firebase bağımlılıkları çözülemedi: {dependencyStatus}");
            }
        });
    }
}
```

4.3. Veri Çekme (Örnek: Çengel Bulmaca Kategorileri)

```
using Firestore;
using System.Threading.Tasks;
using UnityEngine;
using System.Collections.Generic;

public class CrosswordLoader : MonoBehaviour
{
    public async Task<List<CrosswordCategory>> LoadCategories()
    {
        if (FirebaseManager.db == null)
```

```

        {
            Debug.LogError("Firestore başlatılmamış!");
            return new List<CrosswordCategory>();
        }

        Query query =
        FirebaseManager.db.Collection("crossword_categories");
        QuerySnapshot querySnapshot = await query.GetSnapshotAsync();

        List<CrosswordCategory> categories = new List<CrosswordCategory>
        ();
        foreach (DocumentSnapshot documentSnapshot in
        querySnapshot.Documents)
        {
            // Firestore dökümanını C# objesine dönüştür
            CrosswordCategory category =
            documentSnapshot.ConvertTo<CrosswordCategory>();
            category.Id = documentSnapshot.Id; // Document ID'sini manuel
            ata
            categories.Add(category);
        }

        return categories;
    }
}

// Firestore verisini eşleştirmek için bir C# sınıfı
[FirestoreData]
public class CrosswordCategory
{
    [FirestoreDocumentId]
    public string Id { get; set; }

    [FirestoreProperty]
    public string name { get; set; }

    [FirestoreProperty]
    public int xpPoints { get; set; }

    [FirestoreProperty]
    public string iconUrl { get; set; }
}

```

4.4. Çengel Bulmaca Oyunu Entegrasyonu

1. **Kategori Seçimi:** `crossword_categories` koleksiyonundan tüm kategorileri çekin ve oyuncuya sunun.
2. **Kelime Grubu Yükleme:** Oyuncu bir kategori seçtiğinde, `crossword_word_groups` koleksiyonundan `categoryId`'si eşleşen bir kelime grubunu rastgele veya sırayla çekin.
3. **Oyun Verisi:** Çektiğiniz kelime grubundaki `words` dizisini kullanarak çengel bulmacayı oluşturun.

5. Önemli Notlar

- **Güvenlik Kuralları:** Tüm platformlar için Firestore Güvenlik Kurallarını (Security Rules) ayarlamayı unutmayın. Hangi kullanıcının hangi veriyi okuyup yazabileceğini belirleyin.
- **Maliyet:** Firestore kullanımı, okuma, yazma ve silme sayısına göre ücretlendirilir. Veri çekerken gereksiz okumaları önlemek için sorgularınızı dikkatli oluşturun ve mümkünse **limit** kullanın.
- **Çevrimdışı Destek (Offline Support):** Firestore, mobil ve web için varsayılan olarak çevrimdışı veri desteği sunar. Bu, internet bağlantısı olmadığında bile uygulamanın çalışmaya devam etmesini sağlar. Unity için bu özelliği manuel olarak etkinleştirmeniz gerekebilir.

Bu doküman, mevcut Firebase yapısını diğer platformlara entegre etmeniz için sağlam bir temel sunmaktadır. Başarılar!